

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

Faculty Name: Dr.G.Ayyappan

Student Reg. No:
Name:

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING**ASSESSMENT TOOL 3 - ANSWER DOCUMENT***Comparative Analysis: File Organization, Indexing and Hashing Techniques*

Student Name	N. RAMYA
Register Number	192511403
Course Code	CSA05
Course Name	Database Management Systems
Course Outcome	CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency (BL3, BL4)
Activity Tool	Comparative Analysis (Low)
Assessment Tool Weightage	10%
Faculty Name	Dr. G. Ayyappan

Code of Conduct

I, N. RAMYA (Reg. No: 192511403), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____N.RAMYA_____

Comparative Analysis Answers

Question 1

Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Answer:

Sequential file organization stores records in a physical order based on the value of a chosen key field, so that the file itself is always sorted. Heap (unordered) file organization, on the other hand, stores records in no particular order at all — a new record is simply placed wherever there is free space, most commonly appended to the end of the file.

For data insertion, the heap file is far more efficient. Because there is no ordering constraint to maintain, a new record can be appended directly at the end of the file in constant time, $O(1)$. A sequential file, in contrast, must locate the correct sorted position for the new key and then shift every record that comes after it, giving an average insertion cost of $O(n)$, which makes sequential files expensive to update frequently.

For data deletion, both organizations commonly use a 'tombstone' or deletion marker to avoid immediate physical removal, but the underlying cost differs. In a heap file, marking a record deleted is an $O(1)$ operation with no ordering to preserve. In a sequential file, if physical deletion is required, the remaining records after the deleted one must be shifted left to close the gap, again costing $O(n)$ in the worst case.

For data retrieval, the situation reverses. A sequential file allows binary search on the sorted key, giving $O(\log n)$ lookup performance, which is a major advantage for read-heavy systems. A heap file offers no such shortcut; every lookup requires a full linear scan of the file in the worst case, costing $O(n)$.

Overall, heap file organization suits write-intensive workloads such as transaction logs or staging areas where data arrives continuously and is rarely queried by key, whereas sequential file organization suits read-intensive, batch-oriented systems such as payroll processing or report generation where the file is built once and searched or scanned many times afterward.

Sequential File vs Heap File Organization

Aspect	Sequential File Organization	Heap File Organization
Physical Order	Records stored in sorted key order	Records stored in arrival order (no order)
Insertion Cost	$O(n)$ — requires shifting to maintain order	$O(1)$ — new record simply appended
Deletion Cost	$O(n)$ — gap must be closed by shifting	$O(1)$ with tombstone marking
Retrieval Cost	$O(\log n)$ using binary search	$O(n)$ using linear scan
Best Suited For	Read-heavy, batch-processed data	Write-heavy, frequently updated data

[End of Answer 1]

Question 2

Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Answer:

A clustered index determines the actual physical order in which data rows are stored on disk. Because the data file itself is sorted according to the index key, a table can have only one clustered index — for example, in SQL Server or MySQL (InnoDB), a table's PRIMARY KEY column is typically implemented as a clustered index, so an EMPLOYEE table clustered on emp_id physically stores rows in ascending order of emp_id.

A non-clustered index, by contrast, is a separate structure from the actual data. It stores the index key values along with pointers (row locators) back to where the corresponding rows actually reside in the data file, which itself keeps its own separate physical order (often the clustered order, or unordered if no clustered index exists). Because it is just a lookup structure, a table can have many non-clustered indexes.

For example, in a STUDENT table clustered on roll_no, a non-clustered index could additionally be created on the 'email' column to speed up lookups by email without changing the physical row order, which remains sorted by roll_no.

In terms of performance, retrieving a range of rows via a clustered index is extremely fast because the matching rows are physically adjacent on disk, requiring minimal disk head movement. A non-clustered index lookup, however, typically requires an extra step: first search the index to find the row pointer, then jump to the actual data page — this indirection is called a 'bookmark lookup' and adds overhead, especially for range queries that touch many rows.

In terms of maintenance cost, clustered indexes are more expensive to update because inserting or updating a key value may require physically relocating rows to preserve sorted order, whereas non-clustered indexes only need their smaller separate structure updated, leaving the underlying data file untouched.

Clustered Index vs Non-Clustered Index

Aspect	Clustered Index	Non-Clustered Index
Physical Data Order	Determines actual row storage order	Independent of physical row storage
Number per Table	Only one per table	Multiple allowed per table
Example	PRIMARY KEY on emp_id in EMPLOYEE table	Secondary index on email in STUDENT table
Range Query Speed	Very fast — matching rows are adjacent	Slower — needs bookmark lookup to data
Update/Insert Cost	Higher — may relocate rows	Lower — only index structure updated

[End of Answer 2]

Question 3

Compare primary indexing and secondary indexing based on storage requirements and search performance.

Answer:

A primary index is built on the field that also determines the physical sorted order of the data file — typically the primary key. Because it aligns with the file's ordering, a primary index is usually implemented as a sparse index, storing only one index entry per data block rather than per record, since within a block a short scan is enough to find the exact record once the block is located.

A secondary index, on the other hand, is built on a non-ordering field — one that does not determine the physical arrangement of records. Because the data file is not sorted with respect to this field, the index cannot skip records the way a primary index can; it must be a dense index, containing one entry for every single record, so that any value can be located directly without scanning.

In terms of storage requirements, a primary (sparse) index is significantly smaller, since its size is proportional to the number of blocks (records / blocking factor) rather than the number of records. A secondary (dense) index is much larger because it stores one entry per record, which increases storage overhead proportionally to the table size and may require additional disk blocks just to hold the index itself.

In terms of search performance, a primary index generally performs a fast binary search over the smaller sparse index followed by a short intra-block scan. A secondary index also uses binary search but over a much larger structure, and because the underlying data is unordered with respect to the secondary key, each retrieved record may require a separate random disk access, making bulk secondary-key retrieval considerably more expensive in terms of I/O.

For example, in an EMPLOYEE table physically ordered by emp_id, a primary index on emp_id is compact and fast, whereas a secondary index built on the 'department' column, which does not match the physical order, must be dense and typically incurs more disk I/O when retrieving many employees from the same department.

Primary Indexing vs Secondary Indexing

Aspect	Primary Indexing	Secondary Indexing
Built On	Field matching physical sort order	Non-ordering field
Index Type	Usually sparse (one entry per block)	Must be dense (one entry per record)
Storage Requirement	Lower — proportional to block count	Higher — proportional to record count
Search Performance	Fast: binary search + short block scan	Fast lookup but more scattered data I/O
Example Field	emp_id (primary key, sorted order)	department (non-ordering attribute)

[End of Answer 3]

Question 4

Analyze the differences between static hashing and dynamic hashing in database systems.

Answer:

Static hashing uses a fixed number of buckets, N , determined at the time the file is created. The hash function typically computes $h(\text{key}) \bmod N$ to decide which bucket a record belongs to. As long as the number of records stays close to the value the design anticipated, static hashing delivers excellent, near-constant time performance for insertion, deletion, and search.

The major weakness of static hashing is that it does not adapt well to a growing or shrinking dataset. If the number of records grows well beyond what N buckets were designed for, buckets begin to overflow heavily, chains grow long, and performance degrades toward linear search. Recovering from this requires a full reorganization: creating a new, larger set of buckets and rehashing every single record, which is an expensive $O(n)$ operation that also causes the file to become briefly unavailable.

Dynamic hashing techniques, such as extendible hashing and linear hashing, were designed specifically to solve this problem. They allow the number of buckets to grow or shrink incrementally as data is inserted or deleted, without ever needing to rehash the entire file at once. Extendible hashing achieves this using a directory of pointers to buckets that can double in size, while linear hashing grows buckets one at a time in a predetermined round-robin order, both avoiding a full-file reorganization.

The trade-off is that dynamic hashing introduces additional complexity and a small amount of extra overhead — for example, extendible hashing requires maintaining a directory structure, which itself may need to be read from disk or memory, adding one extra level of indirection compared to the direct addressing of static hashing.

In summary, static hashing is preferable when the size of the dataset is known in advance and expected to remain stable, while dynamic hashing is preferable for databases where the data volume is unpredictable or expected to grow substantially over time, since it avoids costly full-file reorganizations.

Static Hashing vs Dynamic Hashing

Aspect	Static Hashing	Dynamic Hashing
Number of Buckets	Fixed at file creation time	Grows/shrinks incrementally with data
Handling of Overflow	Degrades badly; needs full rehashing	Handled gracefully via directory/splits
Reorganization Cost	Expensive, $O(n)$, whole file rehashed	Cheap; only affected bucket(s) restructured
Implementation Complexity	Simple	More complex (directory or linear pointer)

Aspect	Static Hashing	Dynamic Hashing
Best Suited For	Stable, predictable dataset size	Growing or unpredictable dataset size

[End of Answer 4]

Question 5

Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Answer:

Both B-Tree and B+ Tree are balanced, multi-way tree structures where every leaf appears at the same depth, guaranteeing $O(\log n)$ search performance. The key structural difference is where data is stored: in a B-Tree, records (or record pointers) can be stored in both internal nodes and leaf nodes, whereas in a B+ Tree, all actual data is stored exclusively in the leaf nodes, and internal nodes hold only routing keys used purely to guide the search path.

Because B+ Tree internal nodes do not need to store actual data, more routing keys fit into a single disk block, giving B+ Trees a higher fan-out (more children per node) than a B-Tree of the same node size. A higher fan-out means a shorter tree for the same number of records, which translates directly into fewer disk I/O operations per search — a critical advantage for large-scale databases where each level of the tree may require a separate disk access.

B+ Trees also link every leaf node to its neighbouring leaf in sorted order, forming a linked list across the bottom of the tree. This makes range queries extremely efficient: once the first qualifying leaf is located, the rest of the range can be retrieved by simply following 'next' pointers, without needing to re-traverse the internal structure. A plain B-Tree has no such linking, so a range query may require repeated in-order traversal logic across multiple branches.

A B-Tree can, in principle, offer slightly faster access to a specific key that happens to be stored in an internal node, since the search can terminate early without descending all the way to a leaf. However, this benefit is minor and inconsistent, whereas the B+ Tree's advantages in fan-out and range-query support are consistent and substantial at scale.

For these reasons, essentially all major commercial and open-source relational database systems — including MySQL's InnoDB, PostgreSQL, Oracle, and SQL Server — use B+ Tree (or close variants) rather than a plain B-Tree for their primary indexing structures, particularly for large-scale applications where both fast point lookups and efficient range scans (e.g., 'WHERE salary BETWEEN X AND Y') are required.

B-Tree vs B+ Tree Indexing

Aspect	B-Tree	B+ Tree
Data Storage Location	Internal nodes and leaf nodes	Leaf nodes only
Fan-out per Node	Lower (data occupies node space)	Higher (nodes hold only routing keys)
Range Query Support	Requires traversal across branches	Very efficient via linked leaf nodes
Tree Height for Same Data	Taller for same block size	Shorter due to higher fan-out
Real-world Usage	Less common in modern RDBMS	Standard in InnoDB, PostgreSQL, Oracle

[End of Answer 5]

Question 6

Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Answer:

Extendible hashing manages growth using a directory of pointers to buckets, where the directory size is always a power of two, determined by a global depth value. When a bucket overflows and its local depth equals the current global depth, the entire directory doubles in size so that an additional hash bit can be used to split the overflowing bucket. If the bucket's local depth is already less than the global depth, only that bucket splits, and the directory does not need to grow.

Linear hashing takes a fundamentally different approach: it does not use a directory at all. Instead, buckets are split in a strict, predetermined round-robin order controlled by a 'split pointer,' regardless of which specific bucket actually overflowed. When any bucket overflows, an overflow chain is temporarily attached to it, but the bucket pointed to by the split pointer — not necessarily the overflowing one — is the one that actually splits and advances the pointer forward.

In terms of scalability, extendible hashing's directory doubling is a relatively cheap operation, since it only duplicates pointers rather than moving data, and lookup for any key is a very fast, direct two-step process: index into the directory using global-depth bits, then access the bucket. However, if the directory itself grows very large, it may no longer fit entirely in main memory, adding an extra disk access.

Linear hashing avoids maintaining any directory structure at all, which keeps its memory overhead more predictable and avoids the possibility of a large directory needing its own disk access. Its trade-off is that because splitting follows a fixed round-robin order rather than always splitting the bucket that actually overflowed, some buckets may temporarily rely on overflow chains even while others have unused capacity, which can lead to slightly less balanced space utilization in the short term.

Both techniques avoid the full-file reorganization required by static hashing, making them well suited to dynamic, growing databases, but extendible hashing generally offers more consistent per-lookup performance at the cost of directory memory, while linear hashing offers simpler, directory-free implementation at the cost of potentially uneven short-term bucket utilization.

Linear Hashing vs Extendible Hashing

Aspect	Linear Hashing	Extendible Hashing
Directory Structure	None — no directory maintained	Directory of pointers to buckets, doubles as needed
Which Bucket Splits	Bucket at the split pointer (round-robin)	The specific bucket that overflowed
Overflow Handling	Temporary overflow chains permitted	Immediate split; overflow chains rare

Aspect	Linear Hashing	Extendible Hashing
Memory Overhead	Lower — no directory to store	Higher — directory may need extra disk access
Lookup Path	Compute address via split-pointer rules	Direct: directory index then bucket access

[End of Answer 6]

Question 7

Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Answer:

A dense index maintains exactly one index entry for every record in the data file, storing a key-pointer pair for each and every record. A sparse index, by contrast, maintains only one index entry per data block, typically pointing to the first record of that block, and relies on the fact that the data file is sorted so that a short scan within the block is sufficient once the block is located.

In terms of memory usage, the difference is substantial. A dense index's size grows in direct proportion to the number of records, while a sparse index's size grows in proportion to the number of blocks, which is the record count divided by the blocking factor. For a file with a blocking factor of 100, a sparse index would be roughly 100 times smaller than a dense index built on the same data, making it far more likely to fit entirely in main memory.

In terms of access speed for a single lookup, a dense index has a slight edge because it points directly to the exact record with no further scanning needed. A sparse index requires one extra step: after finding the correct block via the index, a short linear scan within that block (bounded by the blocking factor) is required to locate the exact record.

However, because a sparse index is smaller, it is more likely to be fully cached in memory, which can actually make it faster in practice for very large files, since avoiding an extra disk I/O to search a large dense index outweighs the small cost of an in-memory intra-block scan. Dense indexes, if too large to fit in memory, may themselves require additional disk I/O just to search the index.

Dense indexing is generally required whenever the index is built on a non-ordering field (a secondary index), since there is no sorted grouping to exploit. Sparse indexing is only possible on a field matching the physical sort order of the file (a primary index), which is why the choice between dense and sparse is often tied directly to whether the index is primary or secondary.

Dense Indexing vs Sparse Indexing

Aspect	Dense Indexing	Sparse Indexing
Entries Maintained	One entry per record	One entry per data block
Memory Usage	High — proportional to record count	Low — proportional to block count
Single Lookup Speed	Slightly faster — direct pointer to record	Slightly slower — needs intra-block scan
Fits in Memory	May not, for very large files	More likely to fit fully in memory
Applicable To	Any field, including non-ordering (secondary)	Only fields matching physical sort order

[End of Answer 7]

Question 8

Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Answer:

Indexed sequential file organization (as used in ISAM — Indexed Sequential Access Method) combines a sorted data file with a separate index that allows both sequential scanning of the entire file in key order and fast direct access to individual records via the index, rather than requiring a full sequential scan for every lookup.

Direct file organization, more commonly known as a hash file, uses a hashing function to compute the storage address of a record directly from its key, without maintaining any sorted order or index structure at all. This gives extremely fast average-case direct access, typically $O(1)$, since the address is computed rather than searched for.

The key advantage of indexed sequential organization is its flexibility: it supports both efficient direct access (via the index, roughly $O(\log n)$) and efficient sequential/range access (by scanning the underlying sorted file or following index pointers in order), making it well suited to applications that need both types of access, such as billing systems that process records sequentially each month but also need to look up individual accounts on demand.

The limitation of indexed sequential organization is that, like any sorted file, it suffers from the overflow problem: as records are inserted, the file periodically needs reorganization to keep the index effective, because insertions beyond the original allocated space are placed in overflow areas which, if allowed to accumulate, degrade access performance over time and require periodic rebuilding.

Direct (hash) file organization has the opposite trade-off: it offers superior performance for direct key-based access, but it provides no efficient way to retrieve records in sorted or ranged order — a range query such as 'all accounts between 1000 and 2000' would require scanning the entire file, since hashing intentionally scatters records and destroys any notion of key ordering. Therefore, the choice between the two depends heavily on whether an application needs range/sequential access (favoring indexed sequential) or purely fast point lookups (favoring direct/hash organization).

Indexed Sequential File vs Direct (Hash) File Organization

Aspect	Indexed Sequential Organization	Direct File Organization
Access Method	Index + sorted sequential file	Hash function computes address directly
Direct Access Speed	$O(\log n)$ via index	$O(1)$ average case
Range / Sequential Access	Efficient — file is sorted	Inefficient — no key ordering preserved
Maintenance Issue	Overflow areas need periodic reorganization	Bucket overflow needs rehashing (if static)

Aspect	Indexed Sequential Organization	Direct File Organization
Best Suited For	Applications needing both direct & range access	Applications needing only fast point lookups

[End of Answer 8]

Question 9

Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Answer:

Single-level indexing maintains one flat index structure directly over the data file, containing key-pointer pairs, which is then itself searched (usually via binary search) to locate the target record's block. This works well when the index is small enough to fit comfortably in a small number of disk blocks or in main memory.

As the data file grows very large, however, the single-level index itself also grows large enough that it may no longer fit in memory and must be stored across many disk blocks. Searching such a large index still requires binary search, but now each comparison step during that binary search may itself involve a separate disk I/O to fetch a different index block, which increases the overall cost of a single lookup.

Multi-level indexing solves this by treating the first-level index itself as a data file and building a second-level (and if necessary further) index on top of it, recursively, until the top-level index is small enough to fit in a single block or in memory. This creates a tree-like hierarchy of indexes, which is conceptually the same principle that underlies B-Tree and B+ Tree structures.

The benefit of multi-level indexing is a significant reduction in the number of blocks that must be examined for a lookup: instead of a binary search across many index blocks, the search descends through a small, bounded number of levels (proportional to log of the index size, with a large branching factor per level), each requiring only a single block access, giving predictable and low I/O cost even for very large files.

The trade-off is added structural complexity and a small amount of extra storage overhead for the additional index levels, along with slightly more complex maintenance logic during insertions and deletions, since changes may need to propagate upward through multiple index levels. Despite this overhead, multi-level indexing scales far better than single-level indexing and is therefore the standard approach used in real database systems for large tables.

Single-Level Indexing vs Multi-Level Indexing

Aspect	Single-Level Indexing	Multi-Level Indexing
Structure	One flat index over the data file	Hierarchy of indexes, index of an index
Performance on Small Files	Efficient — index fits in memory/few blocks	Unnecessary extra overhead for small files
Performance on Large Files	Degrades — index search needs many I/Os	Scales well — bounded number of block reads
Search Cost	$O(\log n)$ but many disk accesses if large	$O(\log n)$ with fewer, more predictable I/Os
Real-world Analogy	Simple sorted index file	Basis of B-Tree / B+ Tree structures

[End of Answer 9]

Question 10

Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Answer:

For exact-match queries — where the query specifies a precise value for the search key, such as 'find the student with roll_no = 1042' — hashing techniques generally provide the best performance. A well-designed hash function distributes keys evenly across buckets, giving average-case $O(1)$ access: the bucket address is computed directly from the key, and at most a short chain within that bucket needs to be scanned. Indexing techniques such as B+ Trees also handle exact-match queries efficiently, but at $O(\log n)$, which, while fast, is inherently slower than the near-constant-time performance hashing can achieve when collisions are well controlled.

For range queries — where the query asks for all records whose key falls within an interval, such as 'find all orders with order_date between 1 January and 31 March' — the evaluation reverses sharply in favour of indexing, and specifically B+ Tree indexing. Because a hash function intentionally scatters keys pseudo-randomly across buckets to achieve even distribution, there is no relationship between key order and physical/bucket location, so a range query against a hash index would require scanning the entire structure, effectively degrading to $O(n)$ with no benefit from hashing at all.

A B+ Tree, by contrast, is explicitly designed to preserve key order and link its leaf nodes together in sorted sequence, so a range query only needs to locate the starting key ($O(\log n)$) and then follow leaf pointers to collect the remaining matching records, giving a combined cost of roughly $O(\log n + k)$, where k is the number of qualifying records — dramatically better than a hash-based scan for anything but a very narrow range.

This trade-off means that neither technique is universally superior; the right choice depends entirely on the dominant query pattern of the application. Systems that perform mostly single-key lookups (such as a session cache keyed by session ID) benefit more from hash indexing, while systems that regularly perform range-based reporting or sorted retrieval (such as financial transaction histories or date-ranged analytics) benefit far more from B+ Tree indexing.

In practice, most production relational database systems address this by supporting both index types simultaneously and allowing the database administrator or query optimizer to choose the appropriate index per column based on expected query patterns — for example, PostgreSQL offers both a HASH index type for pure equality lookups and a B-Tree (B+ Tree variant) index type as its default, general-purpose choice precisely because it handles both exact-match and range queries reasonably well.

Hashing vs Indexing (B+ Tree) — Query Performance Evaluation

Query Type	Hashing Techniques	Indexing (B+ Tree) Techniques
Exact-Match Query	Excellent — average $O(1)$ direct access	Good — $O(\log n)$, slower than hashing

Query Type	Hashing Techniques	Indexing (B+ Tree) Techniques
Range Query	Poor — degrades to full $O(n)$ scan	Excellent — $O(\log n + k)$ via linked leaves
Key Ordering Preserved	No — keys scattered by hash function	Yes — sorted order maintained throughout
Best Application	Session caches, equality-only lookups	Reporting, sorted retrieval, range filters
Typical DB Support	HASH index type (e.g., PostgreSQL)	Default B-Tree/B+ Tree index (most RDBMS)

[End of Answer 10]